

COMP(2041|9044) 26T2 — Make

<https://www.cse.unsw.edu.au/~cs2041/26T2/>

- Even small software systems need to use tools to control builds.
- Many, many tools available
- Tools popular with developers often changing, and specific to platform/language.
- We'll look at a classic tool **make** which is still widely used e.g. Linux kernel
- If you want current alternatives: cmake + ninja
- But you should know **make**

make allows you to

- document intra-module dependencies
- automatically track of changes for incremental compilation

make works from a file called `Makefile` (or `makefile`)

A `Makefile` contains a sequence of rules like:

```
target : source1 source2 ...  
        commands to create target from sources
```

Beware: each command is preceded by a single **tab character**.

Take care using cut-and-paste with `Makefiles`

Dependencies

The make command is based on the notion of *dependencies*.

Each rule in a Makefile describes:

- dependencies between each target and its sources
- commands to build the target from its sources

Make decides that a target needs to be rebuilt if

- it is older than any of its sources (based on file modification times)

Building Multi-module C Program with incremental compilation

main.c

```
#include <stdio.h>
#include "world.h"
#include "graphics.h"

int main(void)
{
    ...
    drawPlayer(p);
    fade(...);
}
```

world.h

```
typedef ... Ob;
typedef ... Pl;

extern addObject(Ob);
extern removeObject(Ob);
extern movePlayer(Pl);
```

world.c

```
#include <stdlib.h>

addObject(...)
{ ... }

removeObject(...)
{ ... }

movePlayer(...)
{ ... }
```

graphics.h

```
extern drawObject(Ob);
extern drawPlayer(Pl);
extern spin(...);
```

graphics.c

```
#include <stdio.h>
#include "world.h"

drawObject(Ob o);
{ ... }

drawPlayer(Pl p)
{ ... }

fade(...)
{ ... }
```

Building Large C Program

For systems like Linux kernel with 50,000+ files building is either

- inefficient (recompile everything after any change)
- error-prone (recompile just what's changed + dependents)
 - module relationships easy to overlook
(e.g. `graphics.c` depends on a `typedef` in `world.h`)
 - you may not know when a module changes
(e.g. you work on `graphics.c`, others work on `world.c`)

Example Makefile #1

A **Makefile** for the earlier example program:

```
game : main.o graphics.o world.o
    gcc -Wall -o game main.o graphics.o world.o

main.o : main.c graphics.h world.h
    gcc -c main.c

graphics.o : graphics.c world.h
    gcc -c -g -Wall graphics.c

world.o : world.c
    gcc -c -g -Wall world.c
```

Using Make

```
$ make
gcc -c main.c
gcc -c graphics.c
gcc -c world.c
gcc -o game main.o graphics.o world.o
$ make
make: 'game' is up to date.
$ vi graphics.h # change graphics.h
$ make
gcc -c main.c
gcc -o game main.o graphics.o world.o
$ vi world.h # change world.h
$ make
make: 'game' is up to date.
$ make
gcc -c main.c
gcc -c graphics.c
gcc -c world.c
gcc -o game main.o graphics.o world.o
```


How make Works

The make command behaves as:

```
def make(target, dependencies, commands):  
    # Stage 1  
    for each D in dependencies:  
        rebuild D if it needs rebuilding  
    # Stage 2  
    if target does not exist or any dependency is newer than target:  
        # rebuild target  
        run commands
```

Parsing a Makefile in Python

```
def parse_makefile(makefile_name):  
    """return dict mapping makefile targets to (dependencies, build commands)  
    ↪ tuple"""  
    rules = collections.OrderedDict()  
    with open(makefile_name, encoding="utf-8") as f:  
        while line := f.readline():  
            if not (m := re.match(r"^(\S+)\s*:\s*(.*)", line)):  
                continue  
            target = m.group(1)  
            dependencies = m.group(2).split()  
            build_commands = []  
            while (line := f.readline()).startswith("\t"):  
                build_commands.append(line.strip())  
            rules[target] = (dependencies, build_commands)  
    return rules
```

[source code for make.v1.py](#)

Building everything whether needed or not - Implementation in Python

```
def build(target, rules, dryrun=False):
    """recursively check dependencies and, if needed, run commands to build
    ↪ target"""
    (dependencies, build_commands) = rules.get(target, ([], []))
    for d in dependencies:
        build(d, rules, dryrun)
    if not build_commands and not os.path.exists(target):
        sys.exit(f"*** No rule to make target {target}")
    for command in build_commands:
        print(command)
        if not dryrun:
            subprocess.run(command, shell=True)
```

[source code for make.v0.py](#)

How make builds what is needed - Implementation in Python

```
def build(target, rules, dryrun=False):
    """recursively check dependencies and, if needed, run commands to build
    ↪ target"""
    (dependencies, build_commands) = rules.get(target, ([], []))
    build_needed = not os.path.exists(target)
    for d in dependencies:
        build(d, rules, dryrun)
        build_needed = build_needed or os.path.getmtime(d) >
    ↪ os.path.getmtime(target)
    if not build_needed:
        return
    if not build_commands and not os.path.exists(target):
        sys.exit(f"*** No rule to make target {target}")
    for command in build_commands:
        print(command)
        if not dryrun:
            subprocess.run(command, shell=True)
```

[source code for make.v1.py](#)

Make command-line Arguments

If **make** arguments are targets, build just those targets:

```
$ make world.o  
$ make clean
```

If no args, build first target in the **Makefile**.

The **-n** option instructs **make**

- to print what it would do to create targets
- but don't execute any of the commands

A different makefile name can be optionally specified with **-f**

- to print what it would do to create targets
- but don't execute any of the commands

Command-line Arguments - Implementation in Python

```
def main():  
    """determine targets to build and build them"""  
    parser = argparse.ArgumentParser()  
    parser.add_argument("-f", "--makefile", default="Makefile")  
    parser.add_argument("-n", "--dryrun", action="store_true")  
    parser.add_argument("build_targets", nargs="*")  
    args = parser.parse_args()  
    rules = parse_makefile(args.makefile)  
    # if not target is specified use first target in Makefile (if any)  
    build_targets = args.build_targets or list(rules.keys())[:1]  
    for target in build_targets:  
        build(target, rules, args.dryrun)
```

[source code for make.v1.py](#)

Makefile - variables & comments

```
# string-valued variables/macros
CC = gcc
CFLAGS = -g
LDFLAGS = -lm
BINS = main.o graphics.o world.o

# implicit commands, determined by suffix
main.o      : main.c graphics.h world.h
graphics.o  : graphics.c world.h
world.o     : world.c

# pseduo-targets
clean :
    rm -f game main.o graphics.o world.o
    # or ... rm -f game $(BINS)
```

Parsing Variables and comments in Python

```
variables = {}  
with open(makefile_name, encoding="utf-8") as f:  
    while line := f.readline():  
        # remove any comment  
        line = re.sub(r"#.*", "", line)  
        # check for variable definition  
        if m := re.match(r"^s*(\S+)\s*=\s*(.*)", line):  
            variables[m.group(1)] = m.group(2)  
            continue  
        line = replace_variables(line, variables)
```

[source code for make.v2.py](#)

```
def replace_variables(line, variables):  
    """return line with occurrences of $(variable) replaced by variable's  
    ↪ value"""  
    return re.sub(r"\$((.*?)\s*)", lambda m: variables.get(m.group(1), ""),  
    ↪ line)
```

[source code for make.v2.py](#)

Building the Latest Python from Sources with make

```
$ curl -sO https://www.python.org/ftp/python/3.15.0/Python-3.15.0a8.tar.xz
$ tar xf Python-3.15.0a8.tar.xz
$ cd Python-3.15.0a8
$ find . -type f|wc -l
4929
$ find . -type f|sed 's/.*\\.//'|sort|uniq -c|sort
...
$ ./configure
...
creating Makefile
$ make
gcc ...
...
$ ./python
Python 3.15.0a8 (main, Apr 14 2026, 10:42:12) [GCC 14.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

make in parallel

The **-jN** option instructs **make** to build dependencies in parallel using up to N parallel processes

For example an approximately 7x real-time speedup building Python:

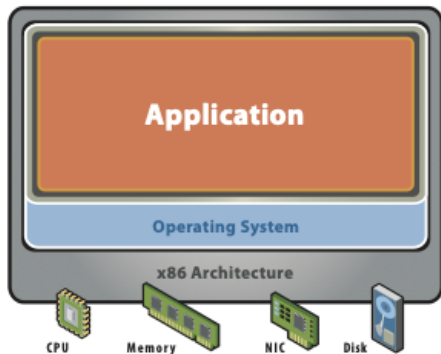
```
$ make clean
$ time make -j16
...
real    0m13.556s
user    1m55.979s
sys 0m7.663s
$ make clean
$ time make
real    1m19.566s
user    1m15.477s
sys 0m4.032s
```

- allows multiple virtual computers, called virtual machines (VMs), to run on a single physical computer.
- each virtual machine behaves as if it has its own:
 - CPU
 - memory
 - storage
 - network interface
 - operating system

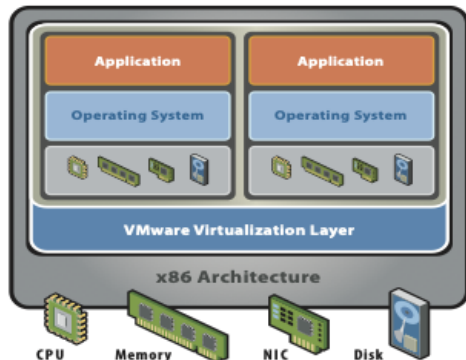
even though all of these resources are actually shared with other virtual machines.

Virtualisation

- separation of resource from the underlying hardware
- an abstraction layer on top of the hardware



- Single OS per machine
- Software and hardware tightly coupled
- Underutilized resources
- Inflexible and costly infrastructure



- Hardware independent of OS and applications
- Virtual machines to any system
- OS and application as a single unit into virtual machines

What is a hypervisor

- A hypervisor is software (or firmware) that creates and manages virtual machines.
- Its responsibilities include:
 - allocating CPU time
 - allocating memory
 - virtualising devices
 - isolating virtual machines *scheduling execution
- It makes one physical computer appear as many virtual computers.
- Examples include Hyper-V, VMware ESXi, Xen, and VirtualBox (hosted hypervisor)

What use virtualisation

Virtualization is one of the key technologies behind cloud computing.

- Better hardware utilisation
 - multiple VMs share the same hardware
 - hundreds of VMs on one physical server in clouds
- Isolation
 - each VM is isolated. If one VM crashes, the others continue run normally
 - malware inside one VM usually cannot affect the others
- Flexibility
 - different operating systems can run simultaneously

- A container is a lightweight, isolated environment that packages an application together with everything it needs to run.
- Why are containers lightweight?
 - Unlike a virtual machine, a container does not contain its own operating system.
 - All containers running on the same machine share the the same operating system kernel
 - A container often starts in less than a second, but a virtual machine may take tens of seconds or even minutes to boot.

- Docker is a platform that creates and manages containers
- Create a Debian Linux container

```
$ docker run --privileged --name linux -it debian:latest bash
```

- `--privileged`: gives the container almost all the privileges of the host
- `--name`: assigns the container a name
- `-it`: gives an interactive shell.
- `debian:latest`: the Docker image to use
- `bash`: starts the Bash shell
- After running this command, it automatically log into the container and the prompt changed to something similar to

```
root@d359e0042f2a: /#
```


Other useful Docker commands

list current running containers

```
docker ps
```

show all containers including those not running

```
docker ps -a
```

#pause/unpause the container

```
docker pause <container_name_or_id>
```

```
docker unpause <container_name_or_id>
```

#start/stop a container

```
docker start/stop <container_name_or_id>
```

#log into a running container

```
docker exec -it <container_name_or_id> bash
```